API de gerenciamento de tarefas com FastAPI e SQLite

Relatório acadêmico de implementação e validação

1 Objetivo e estado da entrega

O projeto implementa uma API REST para gerenciar tarefas fictícias em Python, FastAPI e SQLite. O CRUD foi testado localmente por testes automatizados e pela interface Swagger, incluindo encerramento e reinício real do processo. O código está publicado em https://github.com/shamiryy/api-tarefas-fastapi-sqlite.

A hospedagem pública ainda não foi executada. A criação de recursos Azure aguarda autorização para consumo de créditos acadêmicos e definição de um SKU disponível. Standard_B1s está indisponível nesta assinatura em Canada Central e Central US (NotAvailableForSubscription). URL pública da API: pendente. Documentação interativa hospedada: pendente. Este relatório separa resultados locais de implantação planejada; não comprova funcionamento em nuvem.

Nome completo: pendente. Matrícula: pendente. Turma: pendente. Disciplina: pendente. Professor: pendente. Data de verificação: 02/11/2026; horários dos resultados técnicos em UTC.

2 Linguagem e bibliotecas

Python é uma linguagem de alto nível. As anotações de tipos descrevem os dados e permitem que o FastAPI e o Pydantic validem entradas. FastAPI define rotas HTTP e gera o esquema OpenAPI e a interface Swagger. Uvicorn executa a aplicação pelo protocolo ASGI. O módulo sqlite3 faz parte da biblioteca padrão do Python e acessa um banco em arquivo, sem servidor de banco separado.

Versões efetivamente executadas: fastapi 0.162.0; uvicorn 0.54.0; pydantic 2.14.2; starlette 1.10.0; pytest 9.2.0; httpx 0.28.1; playwright 1.64.1; Python 3.13.7; SQLite 3.50.4. Os arquivos requirements.txt e requirements-dev.txt fixam dependências diretas; requirements-lock.txt registra o ambiente completo usado, inclusive ferramentas de documentação. Use requirements-dev.txt para executar os testes e requirements.txt na implantação. Não é necessário instalar ferramentas de geração do relatório para servir a API.

O código utiliza funções síncronas nas rotas de banco: FastAPI as executa em seu conjunto de threads. Cada operação abre e fecha sua conexão SQLite, com transação e timeout de 10 segundos. A configuração proposta usa uma instância e um worker; isso simplifica o acesso ao arquivo, mas não elimina a serialização das escritas do SQLite.

3 Estrutura e funcionamento

app/main.py contém os modelos de entrada e saída, a fábrica create_app, a criação da tabela no ciclo de vida e as rotas. app/__init__.py identifica o pacote. tests/test_api.py contém testes isolados com diretórios temporários. scripts/backup.py realiza backup consistente e verificação de integridade. scripts/capture_local.py executa a demonstração real e salva prints e o registro do reinício. deploy contém o serviço systemd, Caddyfile, script de instalação e provisionamento preparado. docs contém relatório, diagramas, instruções e evidências.

A variável DATABASE_PATH determina o arquivo SQLite. Sem configuração, o caminho é data/tarefas.db, relativo ao diretório de execução. .env.example é um exemplo e não é carregado automaticamente. Na VM proposta, o caminho absoluto será /var/lib/tarefas/tarefas.db. O Git ignora bancos, backups, .env, chaves, ambiente virtual e arquivos privados. Apenas código, documentação e evidências com dados fictícios foram incluídos.

A aplicação cria a tabela caso ela não exista; não apaga registros na inicialização. Consultas recebem valores por placeholders ?, impedindo a concatenação de dados do usuário no comando

SQL. Operações de escrita são confirmadas ao sair do contexto de transação; exceções causam rollback.

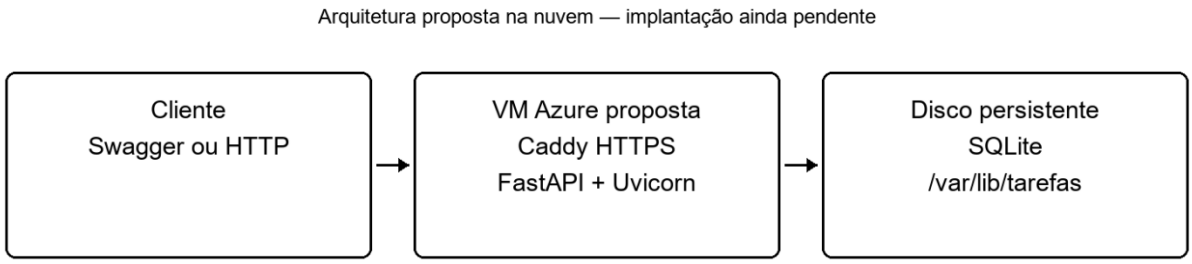


Figura 1. Arquitetura proposta; nuvem ainda pendente.

4 Modelo de dados

A tabela tarefas possui cinco campos. id: INTEGER PRIMARY KEY AUTOINCREMENT, gerado no servidor. titulo: TEXT NOT NULL, entre 1 e 120 caracteres após remoção de espaços nas extremidades. descricao: TEXT NOT NULL, até 2000 caracteres, com padrão vazio. status: TEXT NOT NULL, restrito a pendente, em_andamento ou concluida, com padrão pendente no modelo de entrada. data_criacao: TEXT NOT NULL em ISO 8601, produzida no servidor com timezone UTC e exposta como data e hora no OpenAPI.

O ID e a data não são aceitos no corpo da requisição. Campos desconhecidos são rejeitados. O esquema inclui CHECK para tamanho e valores do status. Não há tabela de usuários, autenticação nem relacionamentos. O PUT substitui os campos editáveis: omitir descricao a redefine para vazio; omitir status o redefine para pendente. A data de criação permanece inalterada.

TAREFAS		
id	INTEGER	PK AUTOINCREMENT
titulo	TEXT NOT NULL	1 a 120 caracteres
descricao	TEXT NOT NULL	0 a 2000 caracteres
status	TEXT NOT NULL	pendente | em_andamento | concluida
data_criacao	TEXT NOT NULL	ISO 8601 em UTC

Figura 2. Campos da tabela implementada.

5 Contrato dos endpoints

POST /tarefas recebe JSON com titulo obrigatório e descricao/status opcionais. Retorna 201 com a tarefa criada e cabeçalho Location apontando para /tarefas/ID. GET /tarefas retorna 200 e lista em ordem de ID, com limit de 1 a 100 (padrão 100) e offset maior ou igual a zero (padrão 0). GET /tarefas/{task_id} retorna 200 com a tarefa ou 404 se o inteiro não estiver cadastrado.

PUT /tarefas/{task_id} recebe o mesmo modelo do POST e substitui todos os campos editáveis. Retorna 200; se a tarefa não existe, 404. DELETE /tarefas/{task_id} retorna 204 sem corpo; uma consulta posterior retorna 404. GET /health executa uma leitura da tabela e retorna 200 com {"status":"ok"}. Erros SQLite durante o atendimento recebem 503 com mensagem genérica sem expor caminho ou SQL.

Entradas inválidas recebem 422 com detalhe de validação, incluindo título vazio, status fora da lista, campos extras, tipo inválido e paginação fora do intervalo. Parâmetro de caminho não inteiro também recebe 422. JSON malformado recebe 422. Um ID inteiro ausente, inclusive negativo sem registro, recebe 404. Métodos sem rota correspondente recebem o tratamento padrão do framework.

A interface Swagger fica em /docs, a documentação alternativa em /redoc e o contrato em /openapi.json. Não há endpoint para baixar o arquivo do banco. A aplicação não serve diretórios como arquivos estáticos.

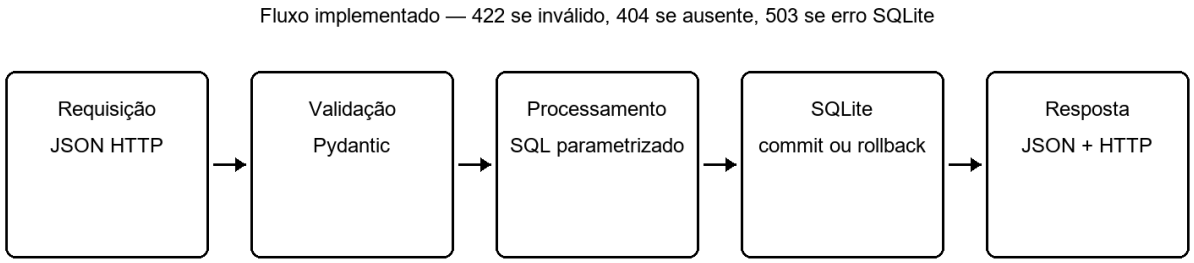


Figura 3. Fluxo de requisição implementado.

6 Exemplos para apresentação

Exemplo de criação: POST /tarefas com Content-Type application/json e corpo {"titulo":"Estudar SQLite","descricao":"Exemplo fictício","status":"pendente"}. Resposta esperada pelo contrato: 201 e objeto contendo esses campos, id gerado e data_criacao UTC. Esse exemplo é ilustrativo; os IDs e horários efetivamente observados estão nos prints e em reinicio-processo-local.json.

Consulta: GET /tarefas/ID. Atualização: PUT /tarefas/ID com {"titulo":"Revisar SQLite","descricao":"Revisão fictícia","status":"concluida"}. Exclusão: DELETE /tarefas/ID. Após excluir, GET /tarefas/ID retorna {"detail":"Tarefa não encontrada"} com 404.

Para uma entrada inválida, envie {"titulo":" "}. O servidor remove os espaços e rejeita o valor vazio com 422. Não coloque nomes, documentos, senhas ou informações reais nas tarefas da demonstração.

7 Execução local

Instale Python 3.13 e Git. Clone o repositório e entre na pasta. No Windows, execute os comandos abaixo no PowerShell. Se uma porta estiver em uso, escolha outra e ajuste a URL no navegador. A porta 8000 estava ocupada neste computador; o script de evidências escolheu uma porta livre automaticamente.

```
git clone https://github.com/shamiryy/api-tarefas-fastapi-sqlite.git
cd api-tarefas-fastapi-sqlite
py -3.13 -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements-dev.txt
$env:DATABASE_PATH="data/tarefas.db"
.venv/Scripts/python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8088 --workers 1
```

Abra http://127.0.0.1:8088/docs. No Swagger, expanda uma rota, clique em Try it out, preencha o corpo ou o ID e clique em Execute. Verifique o código HTTP e a resposta em Server response. Interrompa o servidor com Ctrl+C; execute o mesmo comando novamente e consulte um ID que você criou para confirmar a persistência.

Execute os testes em outro terminal: `.venv/Scripts/python.exe -m pytest -v`. Os testes criam bancos temporários e não usam seu banco de demonstração. Para reproduzir os prints, instale Playwright e Chromium e execute scripts/capture_local.py; essa ferramenta é opcional e não é necessária para a API.

8 Testes e resultados reais

Foram executados 17 casos automatizados: CRUD completo, oito entradas inválidas, três operações em ID inexistente, persistência ao recriar a aplicação, tentativa de injeção SQL como texto literal, atualização inválida preservando dados, paginação inválida e contrato OpenAPI. Resultado: 17 aprovados, zero falhas. Um aviso de depreciação informa que a versão atual de Starlette recomenda httpx2 no TestClient; os testes com httpx terminaram com sucesso. O log original está em docs/evidencias/testes-locais.txt.

Além da suíte, o navegador executou POST, consulta, PUT e DELETE no servidor Uvicorn local real. O script encerrou o processo, aguardou sua saída, iniciou um novo processo com o mesmo arquivo SQLite e comparou todos os campos da tarefa: iguais. Depois da exclusão, a consulta retornou 404. Evidência estruturada: docs/evidencias/reinicio-processo-local.json. Prints identificados como locais acompanham este documento.

O endpoint de saúde não foi usado como prova suficiente do CRUD. Os testes públicos, HTTPS, reinício da aplicação na VM e reinício da VM não foram executados porque a hospedagem não foi criada. Backup/restauração em nuvem também permanece pendente. Backup local com uma tarefa fictícia e restauração em arquivo separado foram aprovados: os dados e a integridade foram conferidos. Evidência em docs/evidencias/backup-local.json.

9 Arquitetura de nuvem e decisões

A proposta usa uma VM Azure Linux para executar Caddy e um serviço systemd com Uvicorn/FastAPI. O cliente acessa HTTPS na porta 443; a porta 80 permite redirecionamento e emissão de certificado. Caddy encaminha a requisição ao endereço de loopback 127.0.0.1:8000. O banco fica fora da pasta do código, no disco de SO gerenciado e persistente. Não se usa disco temporário, armazenamento efêmero de container nem múltiplas réplicas.

VM significa máquina virtual: um computador virtualizado com sistema operacional e recursos de computação. Nós administraríamos o Linux, serviços, atualizações e permissões. Armazenamento persistente conserva os arquivos durante reinicializações e desligamentos; não equivale a backup e pode ser perdido se o disco for excluído. SKU identifica uma oferta ou tamanho específico de recurso, por exemplo Standard_B1s para a VM e StandardSSD_LRS para o tipo do disco.

Uma única instância é suficiente para esta demonstração e evita compartilhar SQLite entre servidores. O SQLite continua limitado em escritas concorrentes e não fornece alta disponibilidade. Uma evolução com múltiplas instâncias exigiria reavaliar o banco, por exemplo PostgreSQL. Essa melhoria não faz parte da implementação solicitada.

App Service Linux foi considerado, mas a documentação oficial alerta para limitações de bloqueio do SQLite no armazenamento compartilhado. Uma VM com disco apropriado oferece controle mais direto desse requisito e atende à menção de VM/SKU do professor. Não foi presumida a existência de uma VM gratuita na conta, nem características internas de plataformas gerenciadas.

10 Inventário e custos da nuvem

Recursos efetivamente criados ou utilizados para hospedar esta API: nenhum. A conta foi consultada, mas isso não constitui hospedagem. Standard_B1s retornou NotAvailableForSubscription em Canada Central; a combinação está bloqueada para esta assinatura. A tabela a seguir registra expressamente os campos ainda não aplicáveis; não confunda nomes propostos com recursos existentes.

Campo	Valor efetivamente verificado
Provedor	Azure, conta consultada; API não hospedada
Serviço e nome de recurso	Nenhum recurso criado para este projeto
Região efetiva	Não se aplica; proposta Canada Central
Plano ou SKU efetivo	Não se aplica; proposta Standard_B1s
vCPU e memória efetivas	Não se aplica; VM não criada
Armazenamento efetivo	Não se aplica; proposta SSD gerenciado E4 de 32 GiB
Sistema operacional efetivo	Não se aplica; proposta Ubuntu Server
Finalidade	Hospedar API e manter SQLite em disco persistente, quando autorizado
Condição da conta	Azure for Students ativa; spendingLimit On; saldo não confirmado
Preço efetivamente cobrado	Não se aplica; nenhum recurso criado

Referência de preços oficiais em Canada Central, verificada em 02/11/2026: VM B1s Linux US$ 0,0116/h; disco E4 LRS US$ 2,64/mês; IP público Standard US$ 0,005/h; operações do disco US$ 0,002 por 10 mil. Para 730 horas, base estimada US$ 14,76/mês, sem operações, eventual saída de dados, tributos e câmbio. É uma estimativa de varejo antes de benefícios; não é cotação contratual nem prova de gratuidade. Dados de origem em docs/evidencias/precos-azure-canadacentral.json e API https://prices.azure.com/api/retail/prices.

Azure for Students anuncia crédito de US$ 100 por 12 meses e serviços gratuitos sujeitos a condições. Não foi verificado o saldo restante ou a elegibilidade individual para a franquia de VM. O limite de gastos está On e não deve ser removido. O próximo passo exige autorização para usar créditos e confirmação do saldo no portal. O processo não contratou recursos pagos nem ativou pagamento por uso.

11 Implantação passo a passo

O procedimento integral e os comandos estão em docs/implantacao.md. Primeiro, confirme saldo, validade, benefício gratuito aplicável e região permitida. Depois de autorizar, execute deploy/provisionar.ps1 com o parâmetro explícito de autorização de consumo de créditos. Ele verifica o limite de gastos e cria apenas um novo grupo exclusivo do projeto. Não modifica outros projetos.

A consulta de SKU retornou NotAvailableForSubscription para Standard_B1s em Canada Central; não crie recursos nessa combinação. Selecione uma combinação permitida com quota e atualize os preços antes de autorizar. A VM proposta usa IP público Standard com DNS Azure. Não abre SSH público: a administração usa Azure Run Command. O script permite somente 80 e 443 no grupo de segurança. Após obter o DNS real, defina API_DOMAIN e execute deploy/instalar.sh por Run Command. Ele instala Python, Caddy, o código do repositório, o ambiente virtual e o serviço

systemd. A unidade executa como usuário restrito tarefas, com escrita apenas em /var/lib/tarefas e reinício em falhas.

Confirme o certificado HTTPS válido no navegador e o status dos serviços. Teste o CRUD pela URL pública, incluindo entradas inválidas e registros inexistentes. Crie uma tarefa, reinicie o serviço e depois a VM, consulte o mesmo ID e compare os campos. Só então preencha a URL real, o inventário de recursos e novas evidências neste relatório e no README.

Os scripts foram preparados; não foram executados em uma VM. Dependências de pacotes Ubuntu, emissão de certificado, quota, disponibilidade do SKU e funcionamento do script em nuvem ainda precisam ser validados. Uma falha deve ser registrada e corrigida antes de marcar a hospedagem como concluída.

12 Backup restauração e manutenção

O script scripts/backup.py usa a API de backup do SQLite e verifica PRAGMA integrity_check. Diferentemente de uma cópia indiscriminada durante uma escrita, essa API produz um snapshot consistente do banco. O script recusa origem inexistente, destino igual à origem e sobrescrita de destino existente.

Exemplo local: `.venv/Scripts/python.exe scripts/backup.py data/tarefas.db .private/backup-2026-11-02.db`. Use um destino privado novo a cada execução. Na VM, adapte para o caminho absoluto e execute como tarefas ou root. Uma cópia no mesmo disco não protege contra exclusão do disco; um backup externo privado depende de escolha e autorização de armazenamento adicional. Não houve contratação desse recurso.

Para restaurar, pare a aplicação, preserve o banco atual com outro nome, copie o backup para o caminho configurado, ajuste as permissões, verifique a integridade e reinicie o serviço. Consulte registros conhecidos e compare os dados. Não restaure enquanto houver escrita ativa. Nunca faça commit do banco ou do backup.

Mantenha o sistema e dependências atualizados após testar compatibilidade. Antes de atualizar o código ou excluir recursos, faça backup. O código cria a tabela inicial, mas não possui migrações automatizadas de esquema. Alterações futuras no modelo exigem migração planejada.

13 Segurança e limitações

Esta é uma demonstração pública sem autenticação. Qualquer pessoa que alcance os endpoints pode criar, consultar, alterar e excluir qualquer tarefa. Não há controle de autoria, autorização, auditoria de usuário ou limitação de requisições. HTTPS protege o tráfego, mas não impede operações indevidas por clientes autorizados pela rede. Use exclusivamente dados fictícios e mantenha a demonstração pública somente pelo período necessário.

As consultas parametrizadas evitam injeção SQL nos valores, mas não resolvem abuso, consumo de disco ou negação de serviço. A validação limita o tamanho dos campos e a listagem, porém não substitui limites de infraestrutura. Uma única VM e um único banco não oferecem alta disponibilidade. Falha do disco sem backup pode causar perda de dados.

Os arquivos .env, chaves, bancos e a pasta .private são ignorados. Capturas foram produzidas com dados fictícios. Não se deve publicar IDs de assinatura, emails pessoais, chaves, tokens ou detalhes financeiros do portal. As limitações são explicitadas para apresentação acadêmica; esta solução não deve receber dados pessoais em produção.

14 Evidências e o que ainda falta capturar

As figuras seguintes são capturas reais de execução local ou do repositório público. Os diagramas são explicações esquemáticas, e não prints de infraestrutura. Os registros técnicos incluem UTC e caminhos relativos verificáveis no repositório.

Permanecem pendentes: Swagger hospedado, CRUD público, tela da VM e seu SKU, disco persistente, plano/condição da assinatura e persistência após reinício na nuvem. Para capturá-los, siga docs/implantacao.md, seção de capturas. Abra VM → Visão geral/Tamanho/Discos e IP público → Configuração. Oculte identificadores pessoais. No Swagger hospedado, capture os códigos 201, 200, 200, 204 e a consulta 404 após exclusão. Para persistência, capture o mesmo ID antes e depois de az vm restart, com data de criação igual.

Não existem prints de nuvem neste documento porque nenhum recurso foi criado. Não substitua essas evidências por diagramas ou por capturas da execução local.

Evidência 1

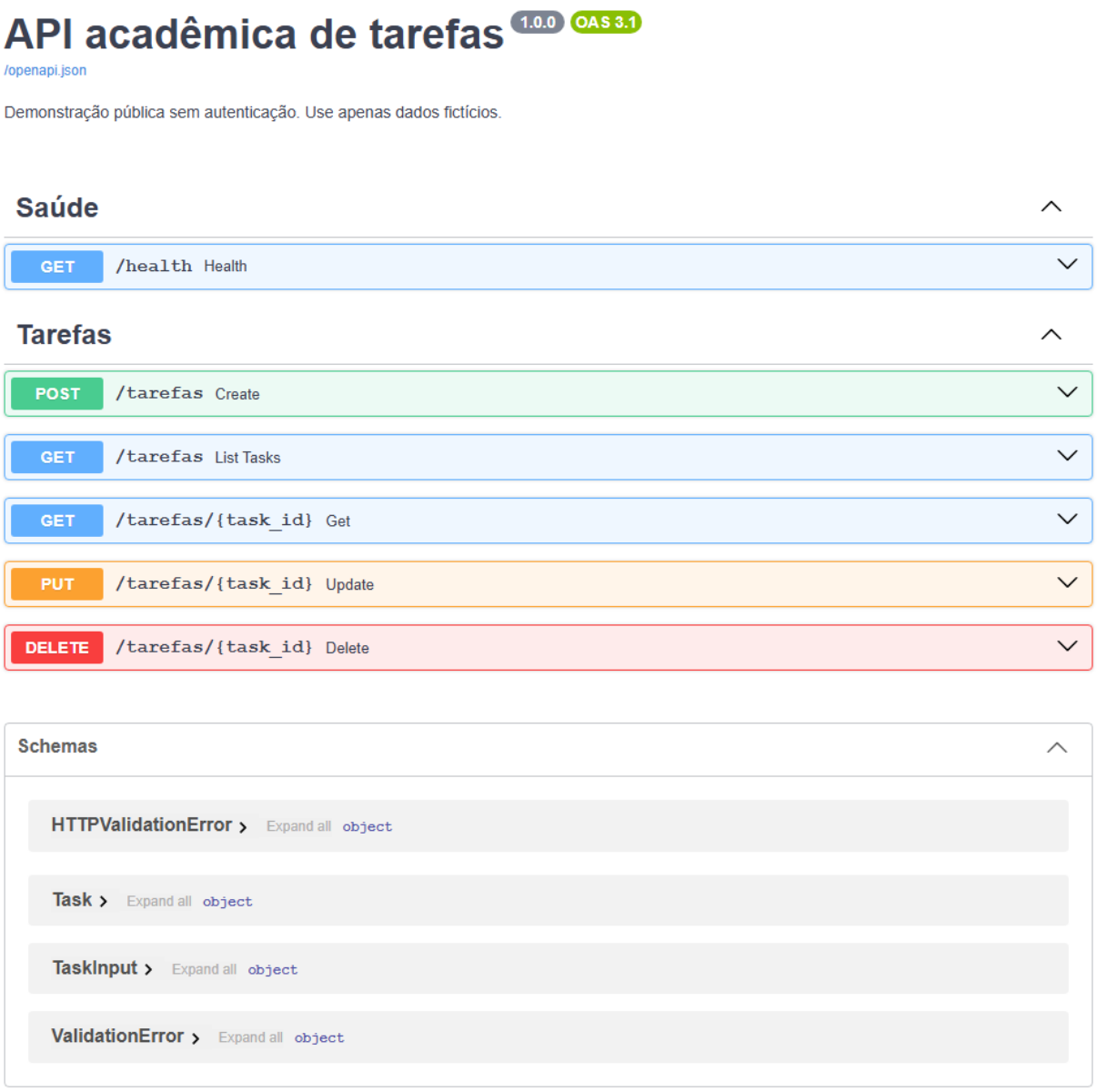


Figura 4. Swagger da API local.

Evidência 2

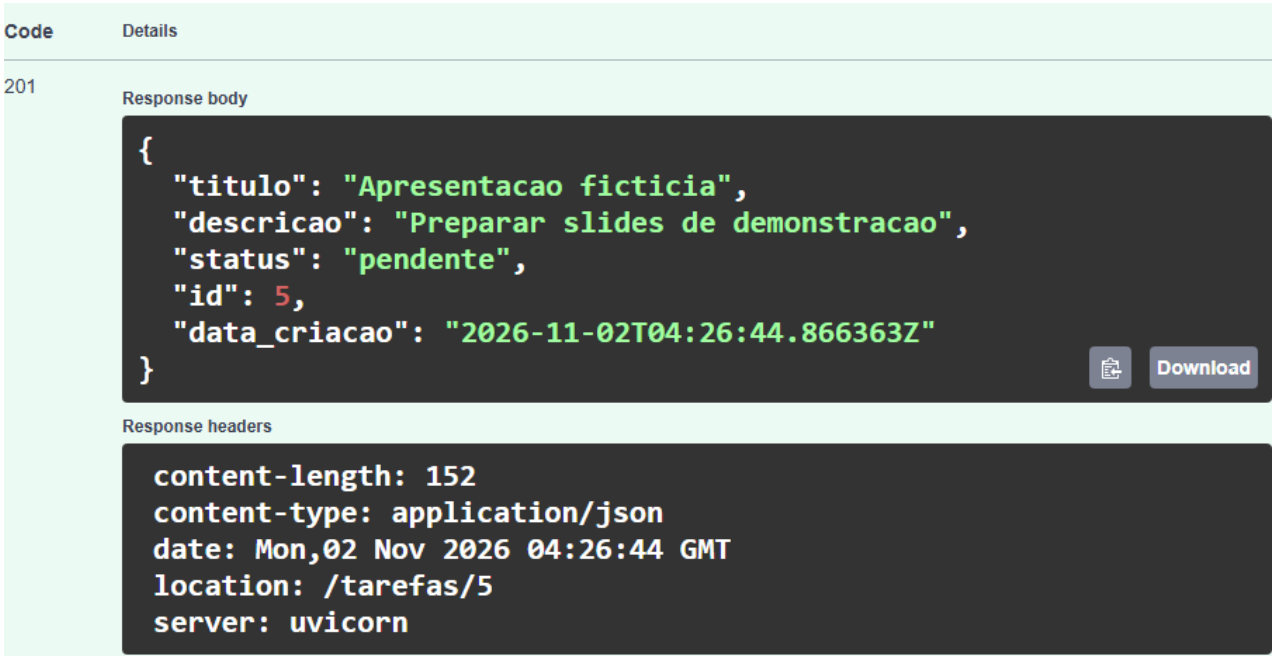


Figura 5. Criação real com HTTP 201 no Swagger local.

Evidência 3

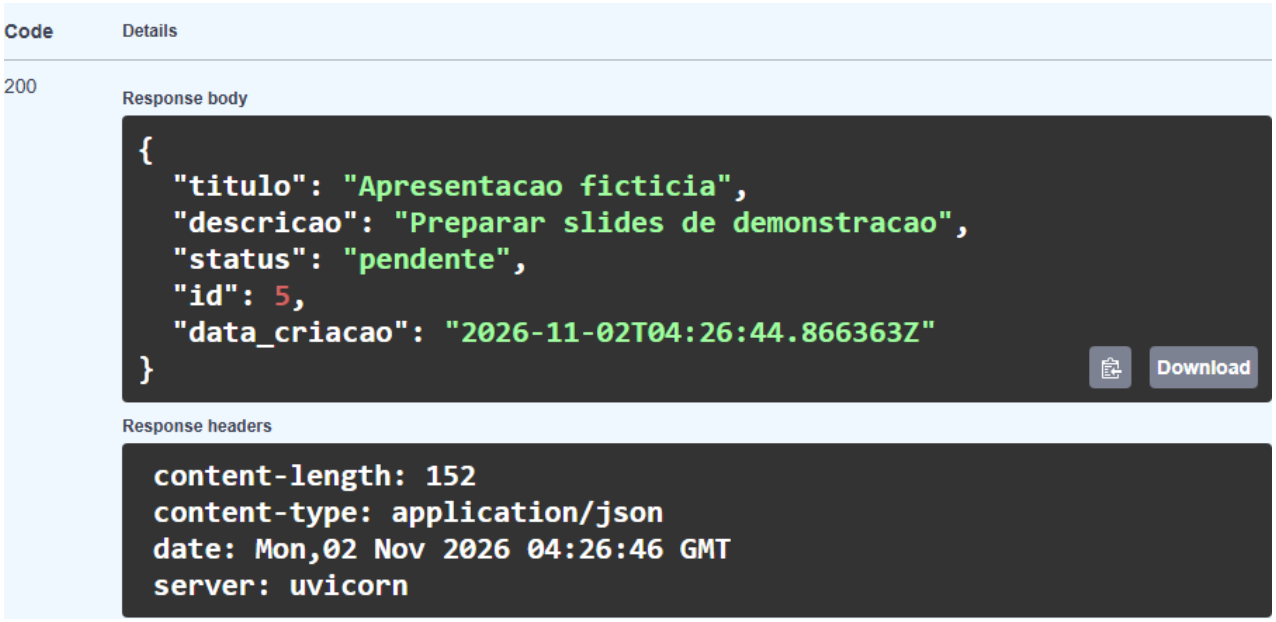


Figura 6. Consulta local da tarefa criada.

Evidência 4



Figura 7. Atualização real no Swagger local.

Evidência 5

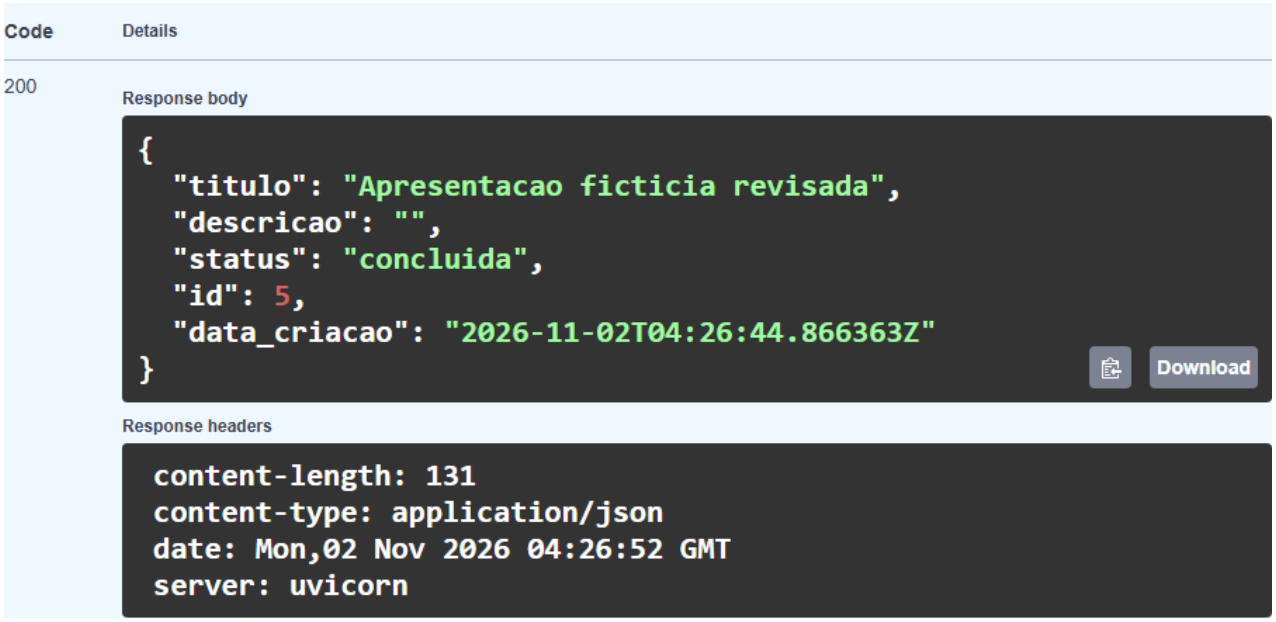


Figura 8. Consulta local após reinício real do Uvicorn.

Evidência 6

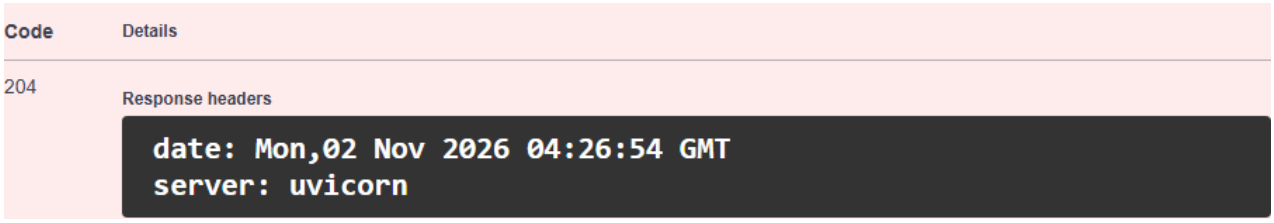


Figura 9. Exclusão real com HTTP 204 no Swagger local.

Evidência 7

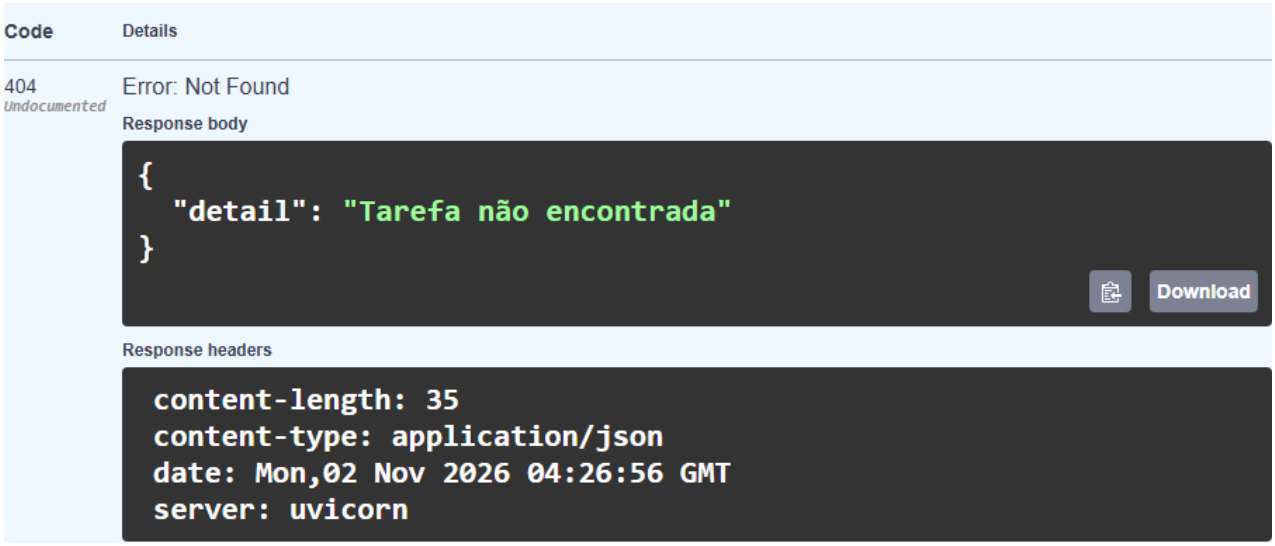


Figura 10. Consulta local após exclusão com HTTP 404.

Evidência 8

```
Caminhos absolutos locais omitidos para privacidade. Resultados preservados.
============================= test session starts =============================
platform win32 -- Python 3.13.7, pytest-9.2.0, pluggy-1.7.0 -- <PROJETO>\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: <PROJETO>
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.16.0
collecting ... collected 17 items

tests/test_api.py::test_crud PASSED                                      [  5%]
tests/test_api.py::test_invalid[data0] PASSED                            [ 11%]
tests/test_api.py::test_invalid[data1] PASSED                            [ 17%]
tests/test_api.py::test_invalid[data2] PASSED                            [ 23%]
tests/test_api.py::test_invalid[data3] PASSED                            [ 29%]
tests/test_api.py::test_invalid[data4] PASSED                            [ 35%]
tests/test_api.py::test_invalid[data5] PASSED                            [ 41%]
tests/test_api.py::test_invalid[data6] PASSED                            [ 47%]
tests/test_api.py::test_invalid[data7] PASSED                            [ 52%]
tests/test_api.py::test_missing[get] PASSED                              [ 58%]
tests/test_api.py::test_missing[put] PASSED                              [ 64%]
tests/test_api.py::test_missing[delete] PASSED                           [ 70%]
tests/test_api.py::test_persistence PASSED                               [ 76%]
tests/test_api.py::test_sql_injection PASSED                             [ 82%]
tests/test_api.py::test_invalid_update_preserves_data PASSED             [ 88%]
tests/test_api.py::test_pagination PASSED                                [ 94%]
tests/test_api.py::test_openapi PASSED                                   [100%]

============================== warnings summary ===============================
.venv\Lib\site-packages\fastapi\testclient.py:1
  <PROJETO>\.venv\Lib\site-packages\fastapi\testclient.py:1: StarletteDeprecationWarning: Using
`httpx` with `starlette.testclient` is deprecated; install `httpx2` instead.
    from starlette.testclient import TestClient as TestClient  # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 17 passed, 1 warning in 2.70s ========================
```

Figura 11. Log original dos 17 testes locais em navegador.

Evidência 9

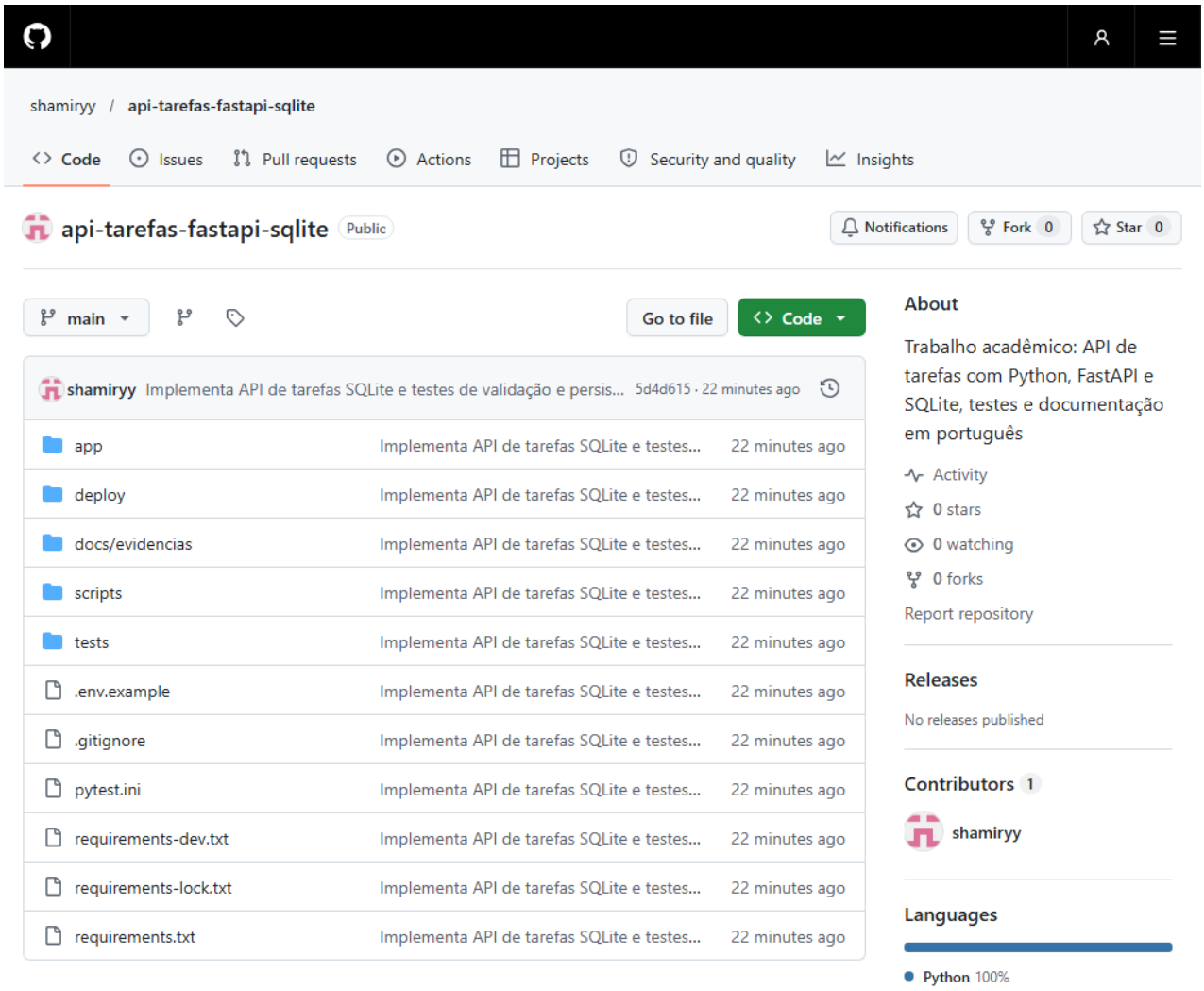


Figura 12. Repositório público publicado no GitHub.

15 Requisitos e pendências

Requisito	Status	Evidência ou ação necessária
API com SQLite e CRUD	Concluído	app/main.py e testes aprovados
Validação e erros HTTP	Concluído	17 casos em testes-locais.txt
Persistência local após reinício	Concluído	reinicio-processo-local.json e print local
Código público no GitHub	Concluído	Repositório publicado e print GitHub
Diagramas renderizados	Concluído	Três PNGs em docs/diagramas
Documentação detalhada	Parcial	Conteúdo local completo; nuvem aguarda implantação
Hospedagem e HTTPS	Pendente	Autorizar créditos, criar VM e validar URL
CRUD e persistência públicos	Pendente	Executar roteiro na hospedagem real
Recursos e SKU efetivos	Pendente	Criar e registrar valores reais do Azure
Prints de nuvem	Pendente	Capturar páginas e respostas indicadas
Identificação acadêmica	Pendente	Preencher nome, matrícula, turma, disciplina e professor

Para concluir a entrega ao professor: autorize o uso de créditos conforme orçamento, confirme saldo e validade sem enviar segredos, execute e valide a implantação preparada, registre o inventário e os prints reais e atualize este documento e o README com a URL comprovada. Preencha os campos acadêmicos. A entrega atual não satisfaz o requisito de hospedagem de ponta a ponta.

16 Encerramento dos recursos

Não há recursos deste projeto para interromper neste momento. Depois da implantação, desalocar a VM interrompe sua computação, mas o disco e o IP podem continuar consumindo créditos. Para evitar custos posteriores, faça backup privado, confirme o conteúdo do grupo exclusivo, exclua o grupo rg-api-tarefas-academica e confirme que az group exists retorna false. Verifique eventuais custos finais no portal. Não exclua o grupo antes da apresentação sem preservar o material necessário.

O comando de exclusão está documentado, mas não foi executado. O limite de gastos da assinatura acadêmica deve permanecer ativo; não é necessário migrar para pagamento por uso para preparar o projeto.

17 Referências técnicas

Fontes oficiais consultadas em 02/11/2026:

FastAPI. SQL Relational Databases: https://fastapi.tiangolo.com/tutorial/sql-databases/

Python. sqlite3: https://docs.python.org/3/library/sqlite3.html

SQLite. Online Backup API: https://www.sqlite.org/backup.html

Caddy. Automatic HTTPS: https://caddyserver.com/docs/automatic-https

Microsoft. Azure for Students: https://azure.microsoft.com/en-us/free/students

Microsoft. About Azure for Students: https://learn.microsoft.com/azure/education-hub/about-azure-for-students

Microsoft. Linux VM Azure CLI quickstart: https://learn.microsoft.com/en-us/azure/virtual-machines/linux/quick-create-cli

Microsoft. Azure Retail Prices API: https://learn.microsoft.com/en-us/rest/api/cost-management/retail-prices/azure-retail-prices

Microsoft. App Service Linux limitations: https://github.com/Azure/app-service-linux-docs/blob/master/Things_You_Should_Know/things_you_should_know.md

Microsoft. Reactivate Azure for Students subscription: https://learn.microsoft.com/en-us/azure/cost-management-billing/manage/azurestudents-subscription-disabled

As páginas sustentam decisões e condições gerais, mas não comprovam saldo ou benefício gratuito individual da assinatura.

18 Roteiro curto para apresentação oral

Minha API gerencia tarefas fictícias: permite criar, listar, consultar, atualizar e excluir. Foi desenvolvida em Python com FastAPI, que valida os dados e oferece documentação Swagger. O SQLite salva os registros em um arquivo persistente, acessado com consultas parametrizadas. Os testes locais passaram, incluindo um reinício real do servidor. O código está publicado no GitHub. A hospedagem Azure ainda está pendente de autorização dos créditos; a proposta é uma VM Linux, um disco persistente para o banco e Caddy para HTTPS. VM fornece computação, disco conserva os registros e SKU identifica o tamanho ou oferta de cada recurso.